

AquaReserve

Smart Supplementary Irrigation System **Design & Testing Document**

MSSE Capstone Project
Quantic School of Business and Technology
Master of Science in Software Engineering

Author and Product Owner: Vishant P
Scope: Simulation Only & Model

Contents

Contents.....	2
Figures.....	3
Tables.....	3
1. Introduction	4
1.1 Purpose of this Document	4
1.2 Problem & Vision	4
1.3 Scope.....	4
1.4 The Farm Scenario	4
1.5 Engineering Methodology.....	5
1.6 Project Deliverables Map.....	6
2. System Overview & Architecture	7
2.1 Closed Simulation Loop.....	7
2.2 Architecture Context.....	7
2.3 Container View.....	7
2.4 Design Principles	8
2.5 Crop Coverage.....	8
3. Architecture & Design Decisions.....	9
4. Software & Architectural Patterns Used.....	13
5. Technology Choices Summary	14
6. Deployment Options & Cost Implications.....	15
6.1 Product (Dashboard + API).....	15
6.2 Modelled Edge System.....	15
7. Results.....	16
7.1 Engine Validation vs pyfao56.....	16
7.2 Controller Comparison across Drought Severity	16
7.3 Reserve-Size Sensitivity.....	17
7.4 Robust (Stochastic) MPC.....	18
7.5 RL Comparison Policy.....	18
7.6 Economics & ROI.....	19
7.7 Study Conclusion.....	20
8. Testing Strategy & Implementation.....	21
8.1 Philosophy & Test Pyramid	21
8.2 Methods.....	21
8.3 Tests Implemented	22
8.4 Quality Gates & CI/CD.....	23

9. Commercialization & Future Work	24
9.1 Commercial Thesis	24
9.2 Hardware Productization & Make vs Buy	24
9.3 The Configurator & Quoting	25
9.4 Commercial Model.....	25
9.5 Demo to Live Product	25
9.6 Installation & Operations.....	26
9.7 Explicitly Out-of-Scope Future Work	26
9.8 What Exists to De-Risk	27
10. Conclusion.....	27

Figures

Figure 1: Closed Simulation Loop.....	7
Figure 2: Architectural Context Diagram	7
Figure 3: Container View.....	7
Figure 4: Full-Farm Production by Controller for 3 Drought Severities @ 20ML Reserve	17
Figure 5: MPC Production vs Reserve Size w/Severe 2019 Drought.....	18
Figure 6: MPC Payback Sensitivity to Crop-Price Swing - $\pm 20\%$	19

Tables

Table 1: Phase-to-Sprint Roadmap.	5
Table 2: Project Deliverables Map	6
Table 3: Container View.....	8
Table 4: Technology Choices by Layer w/Primary Reason.....	14
Table 5: Deployment Options for Dashboard & API w/Indicative Costs.....	15
Table 6: Engine Validation against pyfao56 by Crop	16
Table 7: Controller Comparison - Full-Farm Production vs 3 Drought Severities	16
Table 8: Reserve-Size Sensitivity - Production & Reserve Survival w/Severe 2019 Drought.....	17
Table 9: Robust (Stochastic) MPC vs Point-Forecast MPC & Perfect-Foresight Oracle.	18
Table 10: Economics by Controller - Annual Benefit, Payback & 10Yr NPV.....	19
Table 11: Test Pyramid - Layers, Examples & Tooling.....	21
Table 12: Tests Implemented by Area, Module & Count.....	22
Table 13: Hardware Bill of Materials - Cost Basis & Make vs Buy Decision	24

1. Introduction

1.1 Purpose of this Document

This is the projects required Design and Testing Document. It records, in one place:

1. the design and architecture decisions made;
 - a. the technologies and architectural choices and the reasons for them,
 - b. the software and architectural patterns used and why and
 - c. the recommended deployment options with their relative cost implications,
2. the software testing carried out, including;
 - a. every automated test suite and the reasoning behind each testing method.

It consolidates the project's living design notes, the engine-validation results, the economic model and the comparison and sensitivity study into a single submission-ready reference.

1.2 Problem & Vision

Drought and erratic rainfall threaten crop yield. A finite backup water reserve, harvested rainwater plus a farm dam or bore, can protect crops if it is released intelligently. Moreover, if it is released to the right zone, at the right growth stage. AquaReserve is a software-controlled supplementary irrigation system, delivered as a closed-loop simulation (a digital twin), that schedules a limited reserve across crop zones and growth stages under forecast uncertainty to minimise yield loss per cubic metre of water.

The engineering core is constrained optimisation under uncertainty. Full rainfall replacement is physically infeasible for broadacre cereals because the storage volume required is enormous, so AquaReserve performs precision reserve management rather than full backup. Around that core sit a metrics and cost/ROI layer, a FastAPI service and a React dashboard that make the benefit measurable and the investment case concrete.

1.3 Scope

Scope Guardrail: *This project is simulation only.* No physical hardware is built of the real-world system. The hardware layer - soil-moisture probes, solenoid valves, pump, edge controller and LoRaWAN/MQTT communications - is modelled inside the twin with optional fidelity (sensor noise, drift, latency). The commercialization path that would turn the twin into a physical product is documented as future work in 9. Commercialization & Future Work section and is deliberately out of the graded scope.

1.4 The Farm Scenario

The validation case is a mixed dryland farm in the Mallee near Mildura, Victoria, Australia: 24 hectares across three zones:

- a 4ha onion crop on drip (high-value horticulture),
- 12ha of wheat and
- 8ha of barley (broadacre cereals)

The system is served by a finite 20ML reserve. Weather is driven by real SILO daily FAO-56 data for Mildura Airport (station 76031), with a deterministic synthetic generator as an offline fallback for tests and CI. A normal year (real 2000) and a severe drought (real 2019) anchor the comparisons.

1.5 Engineering Methodology

The project follows Scrum with fixed-length sprints. This project used four sprints to comfortably cover the advanced controllers and the deployed product. The three Scrum roles are kept distinct in practice to exercise the methodology:

- the Product Owner owns the backlog and acceptance,
- the Scrum Master runs planning and review and guards CI and
- the Developer/Code Owner implements and reviews changes.

Every change is via a pull request with green CI before merge to main and produces an auditable history.

Sprint Board Definition of Done: A backlog item is Done when it is implemented behind config (no hard-coded agronomic constants), covered by unit or integration tests with all CI checks passing, documented, merged to main via a reviewed pull request. Then it is accepted by the Product Owner against its acceptance criteria.

Phase to Sprint Roadmap

Table 1: Phase-to-Sprint Roadmap.

Phase	Description	Sprint
0	Repo scaffold; config schema (farm, crops, climate, reserve)	1
1	Weather loader + drought injection; FAO-56 ET; soil-water balance	1
2 / 2b	Crop demand and stress; FAO-33 yield; reserve dynamics; engine validation vs pyfao56 and AquaCrop-OSPy	2
3	Baseline controllers (rainfed/fixed/threshold) + metrics harness	2
4 / 4b	Smart critical-stage controller; EKF then EnKF root-zone estimators	3
5	MPC controller; perfect-foresight oracle upper bound	3
5b	Probabilistic forecasting to robust MPC; RL comparison policy	4
6	Cost / ROI model (capex/opex, payback, NPV, sensitivity)	3
7	Precompute runner to Parquet/DuckDB; FastAPI API; React/TS + ECharts dashboard; deploy	4
8	Full comparison and sensitivity study; finalise design and testing document; demo	4

1.6 Project Deliverables Map

The project outline lists the required components. This table shows where each is satisfied:

Table 2: Project Deliverables Map

Project Requirement	Where it is Satisfied
Software repository with documented code	GitHub repository (shared with quantic-grader); docstrings and per-module docs
Link to the deployed web application	Single Render service (FastAPI serves the built React apps + API); link in the README
Agile task board of user stories and tasks	GitHub Projects Scrum board, linked from the README, spanning four sprints
Design and testing document	This document (design and architecture decisions, patterns, deployment and cost, all testing)
Recorded 15 to 20-minute demonstration	Recorded walkthrough of the deployed dashboard and product app (submitted separately)
CI/CD in active use	GitHub Actions: lint, config validation, tests and coverage across Python 3.10-3.12, plus web builds and a deploy hook

2. System Overview & Architecture

AquaReserve is a closed-loop digital twin of a supplementary irrigation system. It simulates the loop weather to soil-water balance to crop response to reserve dynamics to modelled hardware to controller and back to the soil. The system is time-stepped daily with sub-day resolution for irrigation events. Around the simulation sit a metrics and cost/ROI layer, a FastAPI service and a React + TypeScript dashboard (Apache ECharts) for monitoring and season playback.

The engine is a transparent, control-oriented implementation of FAO-56 and FAO-33 so the estimator (EKF/EnKF) and the controllers (MPC, robust MPC, oracle, RL) have clean state equations to work with. Its correctness is independently validated against the published pyfao56 and AquaCrop-OSPy models. Experiment results are precomputed into a Parquet/DuckDB store and served by the API, so the graded demo never depends on running heavy simulations live on free-tier hosting.

2.1 Closed Simulation Loop

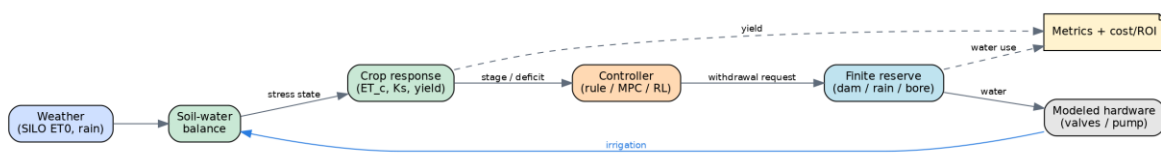


Figure 1: Closed Simulation Loop

2.2 Architecture Context

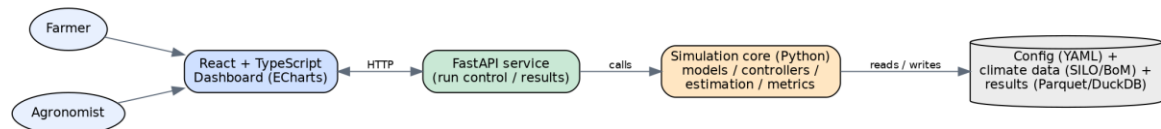


Figure 2: Architectural Context Diagram

2.3 Container View

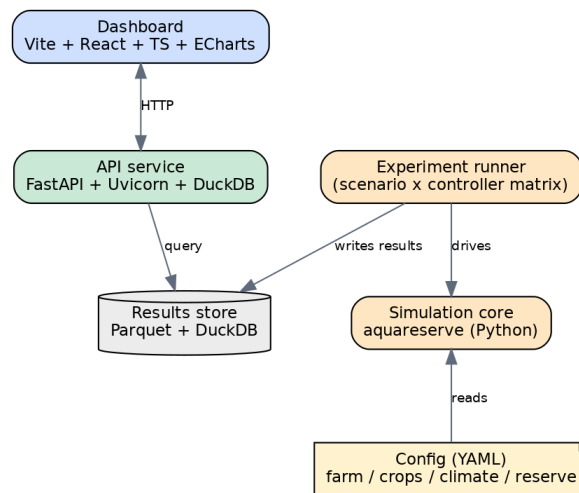


Figure 3: Container View

Table 3: Container View

Container	Technology	Responsibility
Simulation Core	Python Package AquaReserve	Engine, models, estimator (EKF/EnKF), controllers (baseline/rule/MPC/oracle/RL), metrics
Experiment Runner	Python + Parquet	Precompute the scenario x controller matrix into a results store
API Service	FastAPI + Uvicorn + DuckDB	Serve precomputed results and time-series; drive the configurator and the live demo
Dashboard	Vite + React + TypeScript + Apache ECharts	Monitoring, season playback and KPI/comparison views
Product App	Vite + React + TypeScript	Customer configurator and live monitoring demo, behind a login
Results Store	Parquet files + DuckDB	Analytical queries over run results and time-series
Config	YAML under config/	Declarative farm/crop/climate/reserve definitions

2.4 Design Principles

1. **Config-Driven Not Code-Driven:** Every agronomic constant lives in validated YAML; the engine contains logic, not magic numbers. This makes the platform crop-agnostic and experiments reproducible.
2. **Local-First:** Safety-critical control logic must run offline and is never gated by a subscription; cloud features (forecasts, multi-season optimisation, analytics, OTA) are additive.
3. **Strategy-Pluggable Control:** Controllers share one interface so baselines, rule-based, MPC and RL are swapped without touching the simulation loop.
4. **Fail Loud on Bad Input:** Schema validation rejects physically impossible configurations at load time rather than producing silently wrong results.

2.5 Crop Coverage

Because the crop model is a config-driven FAO-56/FAO-33 parameter pack, the system is not limited to the three calibrated crops (onion, wheat, barley). Adding a crop is a data change (one YAML pack), not a code change. The best-fit crops are the high-value and drought-sensitive ones where a small, well-timed volume protects a disproportionate share of yield:

- oilseeds (canola)
- pulses (chickpea, lentil, field pea, faba bean)
- other cereals (oats, triticale, maize)
- horticulture (potato, tomato, carrot and similar)

Perennial vines and trees (winegrapes, almonds, citrus) are a strong future target once the engine models a perennial cycle. Rice and sugarcane fall outside the finite-supplementary paradigm.

3. Architecture & Design Decisions

The design decisions are recorded as concise architecture decision records (ADRs) in the format:

Decision → Context → Reasoning → Consequences

They capture the technology and architectural choices and the reasons for them, which the rubric asks the design document to explain.

ADR-001 – Python for Simulation Core

Decision: Implement the simulation, modelling and control in Python.

Context: Scientific simulation with crop, soil and control models needing a rich ecosystem (NumPy/SciPy/pandas, do-MPC, Gymnasium/SB3, aquacrop).

Reasoning: Unmatched scientific and ML library support; the fastest path from FAO equations to working code; aligns with the owner's sensor-fusion and controls background.

Consequences: Raw numeric loops are slower than C++ - mitigated with vectorized NumPy and the option to drop the edge-control loop to C/C++ later.

ADR-002 – Source Layout Package + Declarative YAML Config

Decision: A src/aquacrop/ package with scenarios defined in config/*.yaml, validated by Pydantic.

Reasoning: src-layout prevents accidental imports of un-installed code and is the modern packaging standard. Pydantic gives schema validation, type coercion and clear errors for free. Separating config from code keeps the platform crop-agnostic and makes experiment sweeps trivial (swap a file, not the code).

Consequences: A small amount of schema boilerplate, repaid immediately by catching invalid scenarios at load time.

ADR-003 – Strategy Pattern for Controllers

Decision: All controllers implement a common observe (state) to per-zone request interface.

Reasoning: The comparison study (rainfed to fixed to threshold to smart to MPC to RL) is the project's core experiment; a uniform interface makes controllers interchangeable and the metrics harness controller-agnostic.

Consequences: Advanced controllers must adapt to the shared interface; worthwhile for a clean comparison.

ADR-004 – Climate Data via Abstracted Loader with Synthetic Fallback

Decision: A loader abstracts the climate source and ships a deterministic synthetic generator for tests and CI.

Reasoning: SILO provides daily FAO-56 ET0 for Australia, ideal for the Mallee validation case, but network access is undesirable in CI. The abstraction lets tests run offline and lets the region be swapped later.

Consequences: Two code paths (real vs synthetic) to keep behaviourally aligned.

ADR-005 – Own Transparent FAO-56/FAO-33 Engine, Validated against Published Models

Decision: Implement our own control-oriented crop-water engine (FAO-56 Penman-Monteith ET with Kc/Ks by stage; FAO-33 Ky yield response) and independently validate it against the peer-reviewed pyfao56 package and the mechanistic AquaCrop-OSPy model on shared scenarios.

Context: The estimator (EKF/EnKF) and the MPC oracle need explicit, differentiable state equations for the soil-water dynamics; a third-party black-box model cannot expose those cleanly.

Reasoning: Owning the engine gives clean state and observation equations for control and estimation, full transparency and reproducibility. Cross-validating against two independent published models provides a credible verification story without sacrificing control integration.

Consequences: We maintain the engine and a validation harness, repaid by control integration and a strong, defensible correctness argument. Alternatives rejected: building on pyfao56 directly (loses control of internal state equations) and using AquaCrop-OSPy as the plant (opaque for control).

ADR-006 – EKF then EnKF for Root-Zone State Estimation

Decision: Fuse noisy and sparse soil-moisture sensors with the soil-water-balance model using an Extended Kalman Filter, then add an Ensemble Kalman Filter and compare estimation error.

Reasoning: EKF is lightweight and transparent and reuses the owner's sensor-fusion portfolio. EnKF is the domain state-of-the-art for assimilating soil moisture into crop/soil models (the literature reports 30-60% root-zone error reductions) and handles nonlinearity via a stochastic ensemble. Implementing both turns control under uncertainty into a concrete, measurable comparison.

Consequences: Two estimators to maintain and tune, isolated in estimation/; EnKF costs more compute per step.

ADR-007 – Perfect-Foresight Oracle & RL Comparison Policy

Decision: Benchmark controllers against a perfect-foresight optimisation oracle (CasADi) that sees the true future weather and a reinforcement-learning policy.

Reasoning: The core claim - smart control preserves yield under uncertainty - needs both a lower bound (baselines) and an upper bound (oracle) to be meaningful. The oracle quantifies the cost of forecast uncertainty; the RL policy provides an AI-engineering technique and an alternative-policy comparison.

Consequences: RL adds training and tuning effort, scoped as a comparison rather than a dependency of the core system.

ADR-008 – Precompute & Serve Results Architecture (Parquet + DuckDB)

Decision: Run the experiment matrix offline into a Parquet/DuckDB results store; the FastAPI service serves those results rather than executing full simulations per request.

Context: Free-tier hosting sleeps and has limited CPU and RAM; the graded demo must be reliable and fast.

Reasoning: Decoupling heavy computation from request time guarantees a responsive, always-working demo; Parquet + DuckDB give fast analytical queries with zero database operations and reproducible results become versioned artefacts.

Consequences: Results are refreshed by re-running the experiment runner. Alternatives rejected: a live-sim backend (timeout risk during grading) and a static-only site (no interactivity).

ADR-009 – Vite + React + TypeScript + Apache ECharts for Dashboard

Decision: Build the dashboard with Vite + React + TypeScript and Apache ECharts.

Reasoning: TypeScript adds the type safety expected of quality front-end code. Vite is the modern fast toolchain. ECharts outperforms lighter React chart libraries on large, multi-series scientific time-series and season playback (canvas/WebGL, brushing, drill-down), which is the dashboard's core interaction.

Consequences: ECharts' imperative option model is slightly less React-native, so it is wrapped behind small typed React components.

ADR-010 – MPC Solved as Linear Program with Scipy (not do-MPC)

Decision: Implement the Model Predictive Controller as a receding-horizon linear program solved each day with `scipy.optimize.linprog` (HiGHS), rather than the do-MPC/CasADi nonlinear framework.

Context: The controller must run reliably in CI and in the graded demo and the soil-water balance has non-smooth kinks (the K_s stress threshold, the drainage clamp) that are awkward for a gradient-based nonlinear solver.

Reasoning: A linear planning model captures the essential allocation trade-off, is convex and always solvable and re-solving daily self-corrects the linearisation and forecast error. Using `scipy` as a core dependency, so there is no install or convergence risk.

Consequences: The plant model inside the optimiser is an approximation of the full nonlinear engine, acceptable because the controller acts on the true engine each day. do-MPC/CasADi remains available for a nonlinear MPC extension.

ADR-011 – Robust (Stochastic) MPC as Quantile Forecast, not Scenario-Recourse LP

Decision: Implement the stochastic MPC as the ADR-010 linear program driven by an upper quantile of an ensemble demand forecast, rather than a multi-scenario two-stage recourse LP.

Context: A natural stochastic-programming form (one shared first-stage decision plus per-scenario recourse) was built first, but with independent per-scenario reserve budgets and free recourse the shared decision became degenerate and true-yield performance collapsed.

Reasoning: A robust quantile posture captures the same intent - hedge against futures worse than the mean - while reusing the proven convex LP unchanged (only the forecast hook differs). Each day it draws an ensemble and plans against the q -quantile of ensemble ET0 (default $q = 0.7$), with rain held at zero. This is fast, always solvable and monotone in the risk knob q .

Consequences: The controller exposes a single interpretable risk parameter. The scenario-recourse LP remains available for future work with a shared cumulative budget and a risk measure such as CVaR.

ADR-012 – RL Comparison Policy via Cross-Entropy Method in NumPy

Decision: Implement the RL comparison policy as a compact linear policy trained by the Cross-Entropy Method (derivative-free policy search) in pure NumPy, evaluating candidates directly on the simulation engine. A Gymnasium environment and an optional Stable-Baselines3 PPO script provide the deep-RL path, but are not part of CI or the default demo.

Reasoning: CEM is a legitimate, well-known policy-search method that needs only NumPy, trains in tens of seconds and is fully seeded and reproducible. It scores each candidate by running the real engine and returning farmgate production value, so the training objective is exactly the deployment objective and the RL

policy is judged on the same engine as every other controller. Trained weights are committed so the demo needs no retraining.

Consequences: The learned policy is competitive with the rule baselines in a normal year but trails the model-based MPC under a real drought - the expected model-free versus model-based gap. The optional PPO path gives a deep-RL data point without burdening CI.

4. Software & Architectural Patterns Used

The rubric asks specifically for the software and architectural patterns used and the reasons for them. The design applies the following, each chosen for a concrete reason rather than for its own sake.

- **Strategy (Controllers):** Every controller implements one observe-to-request interface, so baselines, the smart rule, MPC, robust MPC and RL are interchangeable and the metrics harness stays controller-agnostic (ADR-003).
- **Repository / Loader (Config & Data Access):** Climate and configuration are reached through loaders that hide the source, which lets real SILO data and a synthetic fallback share one interface (ADR-004).
- **Schema-Validation / Value-Object (Pydantic):** Scenario, farm, crop and reserve definitions are Pydantic models used as validated value objects, so impossible inputs fail loudly at load time (ADR-002).
- **Pipeline (the Daily Simulation Step):** Each simulated day flows through a fixed sequence - weather, ET, soil-water balance, crop stress, controller decision, reserve update, yield accumulation - which keeps the loop readable and testable step by step.
- **Dependency Injection by Config:** A scenario object wires the engine's collaborators together from configuration, so swapping a crop pack, a controller or a climate source needs no code change.
- **Precompute & Serve:** An offline experiment runner materialises results into a Parquet/DuckDB store that the API serves read-only, decoupling heavy computation from request time (ADR-008).

5. Technology Choices Summary

Table 4: Technology Choices by Layer w/Primary Reason

Layer	Choice	Primary reason
Simulation Language	Python 3.11+	Scientific ecosystem.
Config / Validation	Pydantic v2	Schema plus clear errors.
Numerics	NumPy / SciPy / pandas	Vectorized models, time-series.
Crop-Water Engine	own FAO-56/FAO-33 (control-oriented)	Clean state equations for control.
Engine Validation	pyfao56 + AquaCrop-OSPy	Independent, published cross-checks.
Estimation	custom EKF + EnKF (NumPy/SciPy)	Transparency, reuse, SOTA comparison.
Control	scipy linprog/HiGHS (MPC, robust MPC, oracle); Gymnasium + SB3 (RL)	Convex, CI-safe, upper bound, AI policy.
Forecasting	empirical ensemble resampling (NumPy)	Probabilistic forecast to robust MPC.
API	FastAPI + Uvicorn	Async, typed, auto-docs.
Results Store	Parquet + DuckDB	Fast analytical queries, zero-ops.
UI	Vite + React + TypeScript + Apache ECharts	Typed, scientific time-series playback.
CI/CD	GitHub Actions	Free, integrates with the repo.

6. Deployment Options & Cost Implications

AquaReserve has two deployment faces:

1. The product (dashboard + API), which is built and deployed.
2. The modelled edge system, which is documented rather than built.

6.1 Product (Dashboard + API)

Table 5: Deployment Options for Dashboard & API w/Indicative Costs

Option	Description	Indicative Cost	When
Single Free-Tier Service (Recommended)	One Render service: FastAPI serves the built React apps + API; precomputed Parquet/DuckDB results on disk	\$0 (Sleeps when idle)	Demo and grading
Split Static + API	React static site on Vercel/Netlify + API on Render	\$0	If faster static hosting is wanted
Small Cloud VM	1-vCPU VM running API + Postgres	Low monthly	Small pilots
Managed Cloud	container service + managed Postgres/TimescaleDB + object storage	Higher monthly	Multi-farm SaaS
On-Premises	farm-office server / NUC	One-off hardware	Air-gapped / local-first preference

Recommendation: Deploy as a single free-tier Render service for the project - one public URL, zero cost and a bulletproof demo because results are precomputed (ADR-008) rather than simulated live. For a real product, the local-first design means the offline control core runs on-prem or on the edge while premium cloud features (forecasts, multi-season optimisation, analytics, OTA) run on managed cloud; the subscription funds the recurring cloud cost while never gating safety-critical local control.

6.2 Modelled Edge System

The physical system that the twin models would use an ESP32-class edge controller running a C/C++ local loop, LoRaWAN field sensors feeding an MQTT gateway and solar plus battery for off-grid resilience. Capital cost is dominated by water storage, then the gateway, then the per-zone nodes; these figures are quantified in the cost/ROI model in 7.6 Economics & ROI section. A buildable tabletop demonstrator of this loop is documented separately in the demo build guide.

7. Results

These results are the payoff of the design decisions above. Every figure is generated by the engine and the experiment runner; none is hand-authored. Production figures are full-farm annual output in tonnes; the reserve is 20ML unless stated.

7.1 Engine Validation vs pyfao56

The transparent engine is cross-validated against the peer-reviewed pyfao56 package (FAO-56 dual crop coefficient), driven by the same real SILO ET0 so the comparison isolates the soil-water and crop-stress logic. Results on the real 2000 Mildura rainfed seasons:

Table 6: Engine Validation against pyfao56 by Crop

Crop	Potential ETc (Kc x ET0)	Actual ET (seasonal)	Depletion Corr	Actual ET Corr
Wheat	+0.0%	-38%	0.93	0.91
Barley	-0.0%	-36%	0.80	0.90
Onion	+0.0%	-18%	0.67	0.93

Potential crop ET matches pyfao56 exactly, confirming the Kc curve and crop-demand computation and daily actual ET tracks strongly (correlation about 0.9). The seasonal actual-ET magnitude is lower because pyfao56 partitions ET into transpiration plus a separate soil-evaporation layer (dual Kc) that keeps evaporating after rain, whereas our control-oriented model uses a single lumped bucket whose soil parameters are calibrated so rainfed cereal yield matches the real Mallee dryland average of about 2t/ha. This is a deliberate, documented methodological difference, not an error; the strong trend agreement validates the dynamics. It is checked continuously in CI against a committed real-2000 SILO fixture (potential ETc within 2%, correlations above 0.85).

7.2 Controller Comparison across Drought Severity

A phase of the project study regenerated the full matrix across three severities (real 2000 normal, a synthetic moderate case, real 2019 severe) times seven controllers times seven reserve sizes. At the 20ML design point every controller roughly triples production over rainfed and the MPC family leads the deployable strategies and approaches the perfect-foresight oracle.

Table 7: Controller Comparison - Full-Farm Production vs 3 Drought Severities

Controller	Normal (t)	Moderate (t)	Severe (t)	Severe Saved
Rainfed (No System)	89.7	46.6	52.2	-
Threshold	257.5	207.2	195.8	+275%
Smart-Stage	256.1	216.9	195.6	+275%
MPC	263.0	230.7	213.0	+308%
Robust MPC	264.4	235.3	211.9	+306%
RL (CEM)	257.1	201.3	184.0	+252%
Oracle (Bound)	264.8	222.5	225.7	+332%

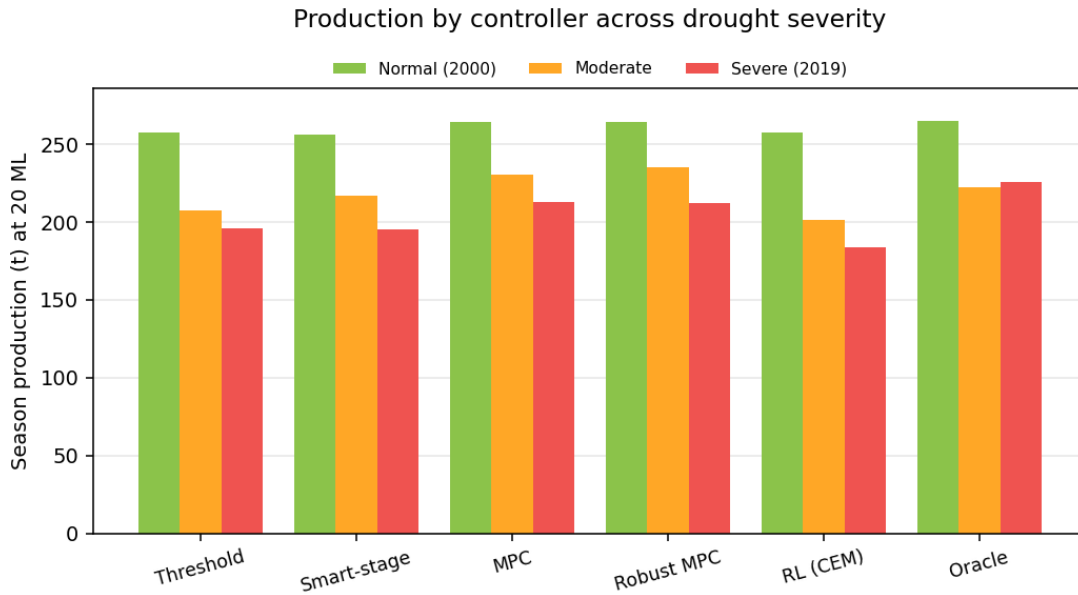


Figure 4: Full-Farm Production by Controller for 3 Drought Severities @ 20ML Reserve

The high-value onion drives the result:

MPC lifts onion from 7.6t/ha rainfed (unmarketable) to 44.0 t/ha (about 88% of full yield, marketable).

Two nuances:

1. Production is not strictly lower under severe than under moderate for every controller, because moderate is a synthetic 30% rainfall cut with elevated demand while severe is the real 2019 year (dry but cool, so low ET0 demand);
2. Robust MPC wins the genuinely uncertain moderate case while tying point MPC under the cool-dry real 2019.

7.3 Reserve-Size Sensitivity

The reserve-size sweep quantifies what storage is worth. Under the real 2019 drought the MPC's production and reserve survival scale with the reserve, with clearly diminishing returns past the 20ML design point:

Table 8: Reserve-Size Sensitivity - Production & Reserve Survival w/Severe 2019 Drought

Reserve	Production (t)	Reserve Survival (Days)
5 ML	183.5	97
10 ML	191.1	100
20 ML (Design)	213.0	121
30 ML	233.4	130
40 ML	241.5	132

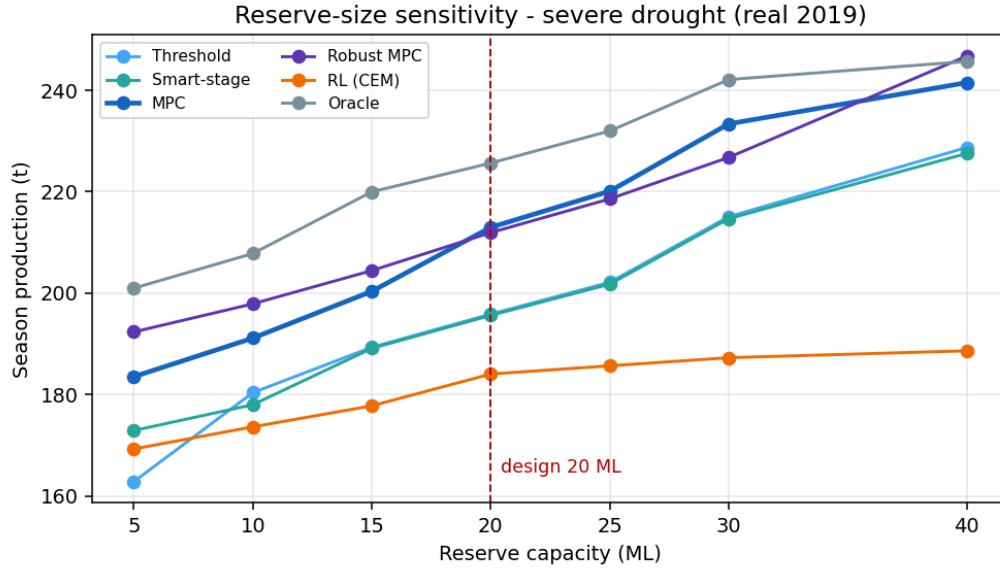


Figure 5: MPC Production vs Reserve Size w/Severe 2019 Drought

Going from 20 to 40ML adds about 28t (roughly 1.4t per extra ML) versus about 30t for 5 to 20ML (roughly 2t per ML). This diminishing return is the evidence behind the software-versus-bigger-dam argument: at some point, smarter control is a cheaper way to buy yield than pouring more concrete.

7.4 Robust (Stochastic) MPC

The robust MPC draws an ensemble of near-term weather trajectories from an empirical resampling forecaster and plans the linear program against an upper quantile of ensemble demand (default $q = 0.7$), hedging against futures where crop water demand runs above the mean.

Table 9: Robust (Stochastic) MPC vs Point-Forecast MPC & Perfect-Foresight Oracle.

Controller	Normal Year (t)	Severe Drought (t)
MPC (Point Forecast)	263.0	213.0
Robust MPC ($q = 0.7$)	264.4	211.9
Oracle (Perfect Foresight)	264.7	225.7

The robust MPC edges the point-forecast MPC in the normal year and stays within about half a percent of it under the 2019 drought - essentially tied. The value of the demand hedge depends on how tight the season is: 2019 was dry but cool, so the reserve already covered the crop. Under a harsher, hotter synthetic drought the same robust MPC matched the perfect-foresight oracle without any foresight. A degenerate multi-scenario recourse LP was implemented and rejected first (ADR-011).

7.5 RL Comparison Policy

The RL agent is a linear policy over eleven per-zone features trained by the Cross-Entropy Method on the real engine and the same real seasons the twin uses. It is competitive with the rule baselines in the normal year but trails both the rules and the model-based MPC under the real 2019 drought - the classic model-free

versus model-based result, made sharper by a real drought regime a compact linear policy must generalise to. The AI policy is competitive, not magic and the domain model earns its keep.

7.6 Economics & ROI

The economic model values each controller's production uplift over rainfed at farmgate crop prices, nets operating and subscription costs and computes payback and NPV. Expected annual benefit blends the normal and severe years by an assumed drought probability. Indicative results (AUD, 10-year horizon, 7% discount, 20% severe-year probability, 126,000 system capex):

Table 10: Economics by Controller - Annual Benefit, Payback & 10Yr NPV

Controller	Expected Benefit / Yr	Payback	NPV (10Yr)
Threshold	~67,000	1.88 yr	~344,900
Smart Rule	~67,000	1.89 yr	~342,200
MPC	~71,000	1.77 yr	~372,900
Robust MPC	~71,000	1.76 yr	~375,400
Oracle (Bound)	~73,000	1.70 yr	~384,000

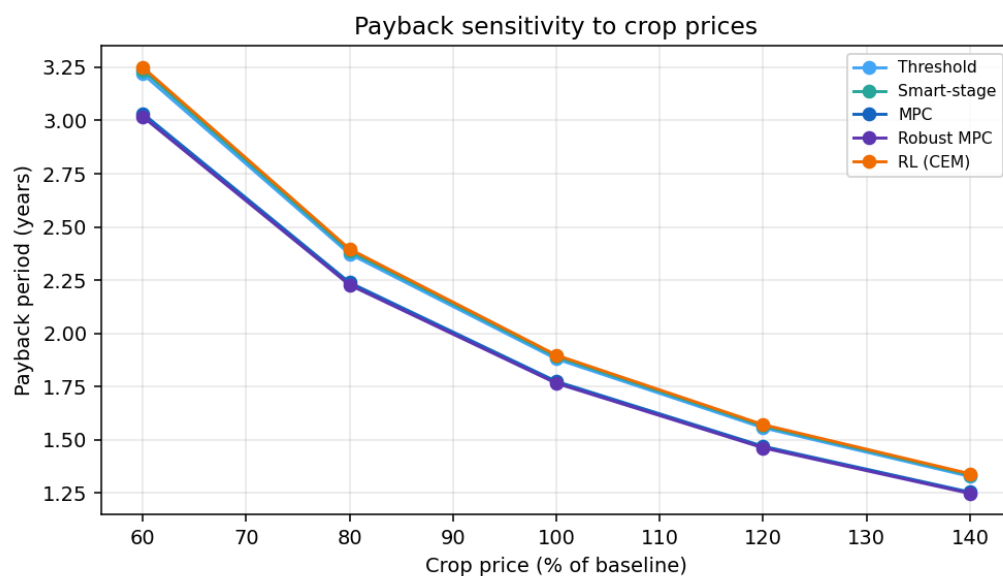


Figure 6: MPC Payback Sensitivity to Crop-Price Swing - $\pm 20\%$

The system pays back in under two years with a strongly positive NPV and model-based control lifts NPV further. The investment case is robust to the assumptions growers worry about most: across a crop-price swing of plus or minus 20% MPC payback moves only between 1.47 and 2.24 years; NPV stays strongly positive even at a 12% discount rate (about AUD 275,000 for MPC); and because the system earns most of its return in drought years, a higher assumed drought frequency shortens payback further.

Software vs Bigger Dam: In the 2019 drought the naive rule needs about a 29ML dam to match the MPC's 213t of output at 20ML, so the control software displaces roughly 9ML of storage worth about AUD 46,000 in avoided dam capex - over a third of the whole system's capital cost. Against a harsher synthetic drought this

reached about 15ML and AUD 77,000, so the dam-substitution value scales with drought severity. All prices and costs are configurable assumptions, not claims.

7.7 Study Conclusion

An MPC-family controller is the recommended deployment. It protects the most yield of any realisable strategy, comes closest to the oracle ceiling and delivers the strongest, most robust economics. The reserve should be sized around 20ML, where the production curve turns over from steep gains to diminishing returns.

8. Testing Strategy & Implementation

8.1 Philosophy & Test Pyramid

Testing has;

- a broad base of fast unit tests (pure functions - unit conversions, ET, soil-water balance, yield, reserve),
- a middle layer of integration tests (config loading and scenario assembly, a full simulation step, controller-in-the-loop) and
- a thin top of end-to-end checks (API to results, a smoke run of the experiment matrix).

All run in CI on every push and pull request across Python 3.10, 3.11 and 3.12.

Table 11: Test Pyramid - Layers, Examples & Tooling

Layer	Examples	Tooling
Unit	Unit conversions, FAO-56 ET vs worked example, soil-water bounds, Ky yield, reserve non-negativity.	pytest
Integration	Example scenario loads and validates; invalid configs rejected; one engine step.	pytest + fixtures
Reference Validation	Our engine vs pyfao56 (ET, soil-water balance) and AquaCrop-OSPy (biomass/yield) within documented tolerances.	pytest + reference outputs
Property / Sanity	Soil water stays within wilting-point and field-capacity bounds; reserve never negative; more water gives at least as much yield (monotonicity).	pytest (parametrised)
Regression	Golden-run metrics for a fixed scenario do not drift unexpectedly.	pytest + stored fixtures
API / End-to-End	The API serves the store; configure, proposal, auth and the live demo endpoints behave.	pytest + FastAPI TestClient

8.2 Methods

- **Validation Tests:** Rejecting impossible soils, reserves and crops - silently-fail with physical inputs and testing at the boundary.
- **Reference-Checked Numeric Tests:** FAO worked examples - keep the agronomy honest.
- **Cross-Model Reference Validation:** Against pyfao56 and AquaCrop-OSPy is the strongest correctness evidence. If the transparent engine tracks two independent published models within tolerance, the results are trustworthy and any divergence is documented and explained.
- **Property & Monotonicity Tests:** Guard invariants that must hold for any input, which point-tests miss.
- **CI across Multiple Python Versions:** Prevents environment-specific breakage and gives graders a green badge as live evidence of working software.

8.3 Tests Implemented

- *The Suite currently Holds 157 Automated Tests*
- *Tests are All Passing*
- *Spanning 26 Test Modules*

Coverage tracks the:

- Engine
- Estimators
- Controllers
- Metrics
- Economics
- API
- Configurator

The table groups the modules by concern.

Table 12: Tests Implemented by Area, Module & Count

Area	Test modules	Tests
Core Engine & Physics	units, et0, soil_water, crop, yield, engine, reserve	42
Weather & Forecasting	weather, silo, forecast	16
Estimation	ekf, enkf	11
Controllers & Control	controllers, mpc, stochastic_mpc, oracle, rl	28
Metrics & Economics	metrics, economic	9
Config, CLI & Validation	config, cli, validation	17
Experiment Matrix	matrix	4
API, Configurator & Demo	api, configurator, demo_bridge	30

The API suite exercises the read-and-serve endpoints against a real precomputed store and covers the authenticated product endpoints end-to-end: **registration and login, the configurator, the proposal PDF, save-a-lead, the admin-only view** and the **live demo decision endpoint** (including that it waters when dry, stops when the reserve is empty and raises its trigger during a critical growth stage). The demo-bridge suite covers the standalone software-in-the-loop companion used by the tabletop demonstrator.

8.4 Quality Gates & CI/CD

Continuous integration runs on GitHub Actions on every push and pull request to main and develop and a green build is required to merge. The pipeline has three jobs:

1. **Lint and Test:** Across a Python 3.10, 3.11 and 3.12 matrix: ruff (lint, style, imports), a config-validation step (`python -m aquareserve.cli validate`), a small precomputed results store so the API tests can run, then pytest with coverage.
2. **Build Web Apps:** On Node 20: type-check and build the results dashboard and the customer product app (tsc + vite), so a broken front-end fails CI.
3. **Deploy:** On main only: trigger the Render deploy hook if configured, otherwise rely on Render auto-deploy - the CD half of CI/CD.

The same gates run locally before every commit, so main stays releasable. Optional static type checking with mypy is available for deeper checks.

9. Commercialization & Future Work

Scope Note: This project is simulation only; the following describes how the validated twin becomes a physical product. Some of it is already built such as, the configurator and the product app. The rest is a documented roadmap such as, the pricing, grant and legal items are prompts to validate with an accountant and a lawyer familiar with Australian agriculture and water law. A fuller roadmap with sprint-ready user stories is in the companion PRODUCT_ROADMAP document.

9.1 Commercial Thesis

The optimisation engine and the ROI evidence represent the hard, defensible component and the study shows they are worth real investment. There is an estimated payback figure under two years, a ten-year NPV of AUD 340,000 to 375,000 and control software that displaces roughly 9ML of dam storage worth about AUD 46,000 under the real 2019 drought.

Most of the commercial work is delivery, hardware and packaging around that core, not new research and development or science investment.

Three principles guide it:

1. Keep the farmer's upfront cost small (buy anything commoditised, build only genuine IP and offer pricing that defers capex).
2. Never let safety-critical control depend on the subscription (the edge node keeps the crop safe when comms, cloud or billing are unavailable).
3. Reuse the ROI model as the sales tool, so the same engine that proved the case quotes and designs each site.

9.2 Hardware Productization & Make vs Buy

The system is defined as a bill of materials with a per-farm sizing rule, so it can be quoted and costed. The custom surface is kept deliberately small. This is evidenced by firmware, the optimiser and integration whereby everything commoditised is bought. The cost structure (indicative AUD for planning) is the single source that also drives the configurator and the economics model.

Table 13: Hardware Bill of Materials - Cost Basis & Make vs Buy Decision

Group	Basis	Indicative cost	Make or buy
Per-Zone Kit	per controlled crop zone (probe, latching valve, control node, solar + battery, flow meter, install kit)	4,000 / zone	Mostly COTS; the node is assembled AquaReserve IP
Central Kit	one per site (LoRaWAN gateway, site controller, reserve level + main flow meter, power cabinet)	6,000 / site	COTS + one assembled controller
Reserve Storage	only when a new dam or tank is built (existing dams are zero)	5,000 / ML	Buy (civil works)
Installation	commissioning lump by zone count	6,000-12,000	Buy (installer network)

Group	Basis	Indicative cost	Make or buy
Optional Pump	only when the farm has no existing pump	3,500	COTS
Recurring (Annual)	subscription (optimisation + control) plus maintenance, comms and power	~2,700 / yr	AquaReserve + COTS

Worked Example: The modelled farm has three zones; a new 20 ML reserve, an existing pump (that totals about AUD 126,000). Per-zone 4,000 x 3, central 6,000, install 8,000 and storage 20ML x 5,000. The genuine IP is the assembled control node (ESP32 + LoRa + valve driver running AquaReserve firmware) and the site controller; the probes, valves, gateway, solar, meters and cabinet are all off-the-shelf. Shipping the node in a COTS IP67 enclosure avoids custom PCB tooling until volume justifies it. A certification plan (electrical safety, IP rating, the Australian RCM mark, EMC, the AS923 LoRaWAN band) makes it legal to sell and install and valves and pump default to a safe state with a local fallback policy so a comms or cloud outage never risks the crop.

9.3 The Configurator & Quoting

The sales tool is built and tested as part of this project and system. The configurator takes a short farm profile (zones, crops, areas, whether a reserve and pump already exist and any water-licence cap) and returns a sized system, an estimated drought-year outcome, an indicative quote from the bill of materials and a payback range, all computed by the validated engine and economics model. When a licence cap is below the NPV-optimal reserve, the recommendation is held to the entitlement and the forgone drought-year yield is reported, so the quote respects the farm's legal water limit. It is exposed at POST /api/configure and wrapped in the customer product app as a wizard, with a one-page proposal PDF and a save-a-quote web-form behind the login. This is roadmap items and delivered early and it means a prospect can see the value before a sales call while the business gets a consistent, engine-grounded quote.

9.4 Commercial Model

The barrier to a sale is the capital cost, so the offer is built to defer or shrink it. Three pricing options let a buyer match the model to their cash flow.

Outright capex; an equipment-as-a-service lease (the configurator already returns an indicative monthly figure that amortises the system) or an outcome-share on the water and yield saved.

A free or low-cost monitoring tier lets a farmer start cheap and upgrade to the paid optimisation and control tier and safety-critical control is never gated by either. Where they apply, drought-resilience and water-efficiency grants and rebates offset capex and drop the out-of-pocket cost further.

A unit-economics model (bill of materials plus install cost, gross margin and the farmer's payback) keeps pricing grounded and sustainable rather than guessed.

9.5 Demo to Live Product

The project deliberately stops at precompute and serve. The live product evolves that into a two-way, per-customer system and this is where a proper database (PostgreSQL/TimescaleDB) is warranted over the demo's Parquet/DuckDB. The steps are:

- Multi-tenant accounts with farms, zones and roles.
- A telemetry ingest pipeline (MQTT or LoRaWAN into a time-series store).
- The optimiser running live on real data and issuing valve and pump setpoints with manual override and an audit log.
- A monitoring app with live status, alerts and a phone-friendly paddock view.
- An edge node that keeps irrigating on a sensible local policy when offline and syncs when reconnected.
- Over-the-air firmware updates with remote diagnostics.

The live Monitor demo already in the product app, hardware-free preview of that loop and the tabletop demonstrator proves the same sense-decide-actuate cycle in real hardware.

9.6 Installation & Operations

Selling the whole system, not just a box, means an install and operations pipeline:

- A site-assessment and design tool that extends the configurator into a full proposal.
- A certified installer network (partnering with existing irrigation dealers) so installs scale without hiring crews everywhere.
- A commissioning checklist and customer handover pack so every install is calibrated and documented.
- Support with an SLA, spare-parts logistics and remote monitoring that catches faults before the grower notices.
- Defined warranty and service-contract terms.

9.7 Explicitly Out-of-Scope Future Work

Two phases are commercial and physical actions that cannot be done in software, so they are recorded as future work:

- **Phase A – Real-Farm Pilot:** Recruit one to three pilot farms, install hardware, calibrate the engine to each site, log a full growing season against predicted yield and water use and have an independent agronomist co-sign the claims. This turns simulation into a validated case study. It needs a physical site and a season. The project and system de-risks this by delivering the validated engine, the calibration tooling and the sensitivity study a pilot would confirm.
- **Phase F – Company, IP & Legal:** Incorporation, trademark and brand, IP protection (the optimiser as a trade secret or patent if novel (firmware and data proprietary), clear data-ownership, privacy and liability terms with appropriate insurance and interoperability with farm-data standards (AgGateway ADAPT, ISOBUS). These need a founder, an accountant and a lawyer. The system sets only the starting IP posture – a proprietary licence, with the engine, firmware and data kept proprietary so the moat is protectable later.

9.8 What Exists to De-Risk

- **The Engine & ROI Model:** Become the configurator (built) and the pilot calibration tool.
- **The Precompute & Serve Backend & React Apps:** Become the monitoring app, extended with live telemetry and control.
- **The 2D & 3D Twins:** Become the customer-facing visualisation of a live farm.
- **The Documented Modelled Edge System:** The sensors, valves, gateway, edge controller, becomes the real bill of materials and node, already prototyped by the tabletop demonstrator.

10. Conclusion

AquaReserve delivers a working, tested and deployed digital twin that answers a real question:

Can software make a finite water reserve protect a drought-stressed harvest and is it worth the money.

The transparent FAO-56/FAO-33 engine is independently validated against two published models.

A family of controllers from simple baselines to model predictive control, a robust quantile MPC, a perfect-foresight oracle and a learned RL policy is compared on the same engine under real drought weather.

The economics show a payback under two years that is robust to the assumptions growers care about. The architecture – config-driven, strategy-pluggable, precompute and serve – keeps the system reproducible, extensible and cheap to demonstrate and 157 automated tests across a three-version CI matrix stand behind it. The design decisions and the testing recorded here are what make those results trustworthy.